

Operator overloading in C++:

Operator overloading is an integral part of C++ and is embedded deep within its ecosystem. Libraries such as standard template library (STL), Boost, vmath, glm (both vmath and glm are used in real time rendering) make heavy use of it. In the larger context operator overloading is one of the manifestations of polymorphism but its not a principle way of achieving the same. Therefore, it is not an integral part of all OOP languages. There are fully supportive OOP languages, most notably Java, which provide minimal or no support for operator overloading.

Part I) What is operator overloading?

All modern, high level languages provide built-in types and built-in operators. Boolean, integer, float, strings are amongst those data types whose provision can be assumed in all contemporary languages. As the main purpose of computer is to perform computations on numbers especially arithmetic computations, all languages provide built-in arithmetic operators on integers and floats. Arithmetic operators are provided with same traditional symbols and syntax as in mathematics to ease programmer's expression. For example, after defining integers `a`, `b`, `c` using `int a = 10, int b = 20 and int c,` we can write arithmetic expressions using as `c = a + b; c = a - b; c = a * b; c = a / b; c = a % b` etc.

Any general purpose programming languages cannot anticipate all types that programmer may require and therefore always provide a way to define a custom type. `struct` statement in C, `class` statement in almost all OOP languages illustrate it. The actual story starts now ! What if some custom data types have their own versions of built-in operators. Consider mathematical types such as vector (not to be confused with `std::vector`), determinant, matrices. Definitions of arithmetic operators such as addition, subtraction, multiplication are well defined for such types. There might be some operations on non-mathematical custom types which can be considered as the type's own version of arithmetic operator. For example string is a non-mathematical type. But it is not too far fetched to say that concatenation of two strings is an addition of two strings.

Built-in operators are designed and implemented to work on objects of built-in types. Therefore, question can be characterized as follows: Can we use a built in operator syntax to invoke the operation on the objects of custom types if a built-in operator is well defined for a custom data type or there is an operation on custom data type which can plausibly, within reason interpreted as a version built-in operator in its context? More concretely, let Matrix be a custom type and `m1`, `m2` and `m3` be its objects. Can we use the syntax `m3 = m1 * m2`, instead of `m3 = m1.multiply(m2)` to store the multiplication of `m1` and `m2` (provided they are dimensionally compatible for multiplication) in `m3`? Answer to this question is operator overloading.

Operator overloading allows application of built-in operators on objects of custom classes provided that the definition of operator in the context of custom class is provided by class definition writer in accordance with the language rules.

Lets decipher the definition with the help of example. Consider the custom class Matrix. Let `m1`, `m2` and `m3` be three matrices of the dimension 4x4. We can write `m3 = m1 * m2` to invoke multiplication on `m1` and `m2` provided definition of multiplication of two matrices is provided by class Matrix in accordance with language rules. Language rules typically dictate header of such function. Because the compiler should be able to find definition.

In other interpretation you can consider built-in operator applied on objects of custom class as a software event handler of which must be present in the class in the form that could be recognised by the compiler. In other words, a member function inside class which enables application of built-in operator on objects of custom class, whose header is controlled by the language rule is a callback function.

In yet another interpretation the operator overloading can be logically reached as follows:

There are two entities : data types and operators each which come in two flavours viz. built-in and user defined. We will create four combinations the last one of which will require operator overloading

Combination I) Data Type : Built-in, Operator : Built-in

Responsibility of definition:

Data Type : Language (via keywords char, int, float etc)

Operators: Language (Compiler maps operators to appropriate CPU instructions)

Syntax:

```
int a = 10, b = 20, sum;  
sum = a + b;
```

Combination II) Data Type : Built-in, Operator : user defined

Responsibility of definition:

Data Type : Language (via keywords char, int, float etc.)

Operators: Programmer (via function syntax. Functions are sort of user defined operators)

Syntax:

```
unsigned long long factorial(unsigned int n)  
{  
    if(n == 0)  
        return (1);  
    else  
        return (n * factorial(n-1));  
}  
  
unsigned long long result = factorial(5);
```

Factorial is a valid operator on int not provided by the language. We can define our operator using function and apply it on object of built in type.

Combination III) Data type : User defined, Operator : User defined

Responsibility of definition:

Data Type: Programmer (via class statement)

Operator: Programmer (via member function of class)

Syntax:

```
class Date  
{  
    private:  
        int day, month, year;
```

```

public:
    Date(int dd, int mm, int yy) : day(dd), month(mm), year(yy)
    {
    }

    int getDay() const
    {
        return day;
    }

    void setDay(int newDay)
    {
        day = newDay;
    }
};

void test()
{
    Date myDate(19, 6, 2026);
    myDate.setDay(25);
    int dd = myDate.getDay();
}

```

Combination IV) Data type : User defined, Operator : Built-in

Responsibility of definition:

Data Type : Programmer (via class)

Operator: Programmer (via by defining a function under header specified by the language)

Syntax:

```

class Matrix
{
private:
    int a[3][3];
public:
    Matrix()
    {
        memset((void*)a, 3 * 3 * sizeof(int), 0);
        a[0][0] = a[1][1] = a[2][2] = 1;
    }
    // Assume get set functions for element indexed by i, j
    Matrix operator+(const Matrix& other) const
    {
        Matrix m;
        for(int i = 0; i < 3; ++i)
            for(int j = 0; j < 3; ++j)
                m.a[i][j] = a[i][j] + other.a[i][j];
    }
}

```

```

        return m;
    }
};

void test()
{
    Matrix m1, m2, sum;
    // assume various elements of m1 and m2 are set using set()
    sum = m1 + m2;
    // internally converted to assembly of the following
    // sum = Matrix::operator+(&m1, m2);
    // &m1 is passed to this, m2 is passed to other.
}

```

Part II) Why operator overloading?

As we have discussed before, the operator overloading is not an integral feature of all OOP languages. Language can be a mainstream OOP language without supporting or minimally supporting the operator overloading as illustrated by Java. Then how can we reason about its role in C++?

The Design and Evolution of C++, an excellent book written by Stroustrup states one of his goals of C++'s class statement is that the objects of custom class should be as flexible to use as the objects of built-in class. We all can agree that programmers develop quick comfort over built-in types and operators by combining various operators in a single expression. Without operator overloading such comfort cannot be extended to user defined objects. For example, Java does not support operator overloading and provides BigInt class through JDK. Consider the following code fragment.

```

void test()
{
    BigInt n1 = new BigInt(34535344546356);
    BigInt n2 = new BigInt(346474765345746346);
    BigInt n3 = new BigInt(45743534746346);
    BigInt n4 = n1.add(n2).sub(n3);    // actual method names may be
                                     // different in JDK
}

```

It is evident that more the complex the expression more tedious will be the syntax. Had operator overloading been in place then we could simply write `n4 = n1 + n2 - n3`; Even more complex expression can be written without hassle.